Two dimensionless ratios define operational limits in hierarchical-memory inference

Geunsik Lim[1*]

[1] College of Information and Communication Engineering, Sungkyunkwan University (SKKU), Suwon, South Korea .

Corresponding author(s). E-mail(s): leemgs@g.skku.edu;

Abstract

We formulate hierarchical-memory inference using two dimensionless ratios: fast-memory residency $\boldsymbol{R_C = C_{fast}/W}$ and compute–transfer overlap $\boldsymbol{R_B = T_{comp}/T_{transfer}}$. These ratios define a capacity-limited region ($\boldsymbol{R_C < 0.5}$), an I/O-limited region ($\boldsymbol{R_C \geq 0.5}$, $\boldsymbol{R_B < 1}$), and a coordination-dominated region ($\boldsymbol{R_C \geq 0.5}$, $\boldsymbol{R_B \geq 1}$). We derive irreducible capacity and transfer-cost bounds and release a measurement harness using wall-clock or device-event timing. A compiled dependent-load microbenchmark on one Intel Xeon CPU shows mean access time increasing from 6.8 ns for a 0.5 MiB working set to 169.7 ns for 256 MiB. The increase spans several cache and translation levels and does not identify a discontinuity at $\boldsymbol{R_C = 0.5}$. These measurements illustrate why residency must be reported, but do not establish the labels on accelerators or production workloads.

Keywords: hierarchical memory systems, memory orchestration, AI inference, working-set residency, compute–transfer overlap, reproducible measurement

1 Introduction

Modern foundation models have shifted the performance frontier of AI inference from arithmetic throughput to *data coordination*. When model weights, activations, and key–value (KV) caches exceed accelerator memory, inference latency is governed less by compute and more by how efficiently execution coordinates data movement across a hierarchical memory system spanning device memory, host memory, and secondary storage [1, 2].

Hardware analyses identify memory capacity, bandwidth, and interconnect latency as important constraints on inference [3]. Separating capacity pressure from exposed transfer is therefore useful when reporting and comparing inference systems.

Existing systems reduce memory pressure via CPU/NVMe offloading [4, 5], GPU–CPU swapping [6], and KV-cache management [7]. Their outcomes depend on model size, batch size, hardware topology, and the amount of traffic that remains on the critical path. This motivates a compact description that exposes those conditions before comparing policies.

We answer this question with a regime-based view. Denning's working-set model and thrashing theory [8] identify capacity saturation as a binary event on a *single-tier* system, and the Roofline model [9] expresses performance as $\min(F \cdot I,\, B_{\text{peak}})$ for a fixed workload on a single tier. Neither framework addresses a *multi-tier* hierarchy in which computation, locality, cross-tier transfer, and synchronisation co-exist as distinct, adjustable latency terms. Our accounting model decomposes end-to-end latency into four terms ($T_{\text{comp}} + T_{\text{mem}} + T_{\text{swap}} + T_{\text{sync}}$) and uses two dimensionless ratios to label an operating point. The three operational labels are governed by the fast-memory residency ratio $R_C = C_{\text{fast}}/W$ and the overlap ratio $R_B = B_{\text{slow}} T_{\text{comp}}/D = T_{\text{comp}}/T_{\text{transfer}}$. The labels state whether majority residency is available and whether ideal compute–transfer overlap can hide compulsory traffic; they do not by themselves predict a policy's speed-up.

We make three contributions:

- **Operational formulation.** We separate residency from compute–transfer overlap using two measurable, dimensionless ratios and state an explicit three-label classification rule.
- **Minimal analytical model.** A structural lower bound on the irreducible memory-access and transfer costs, with regime boundaries following directly from the two ratios: $\theta_B = 1.0$ derived as the overlap boundary ($R_B = T_{\text{comp}}/T_{\text{transfer}} = 1$), and $\theta_C = 0.50$ as the majority-residency crossover.
- **Open proof of concept.** A compiled dependent-load CPU probe illustrates working-set sensitivity, and a separate layered-matrix harness exercises the measurement protocol. We distinguish these CPU observations and unexecuted CUDA/XLA paths from accelerator evidence.

Together, the formulation and CPU data provide a reproducible starting point for testing when hierarchical-memory coordination is constrained by residency or exposed transfer. They do not establish effect sizes or strategy rankings on accelerators.

2 Results

2.1 A two-ratio operational model

We describe a hierarchical-memory operating point using

$$R_C = \frac{C_{\text{fast}}}{W}, \qquad R_B = \frac{B_{\text{slow}} T_{\text{comp}}}{D} = \frac{T_{\text{comp}}}{T_{\text{transfer}}}, \tag{1}$$

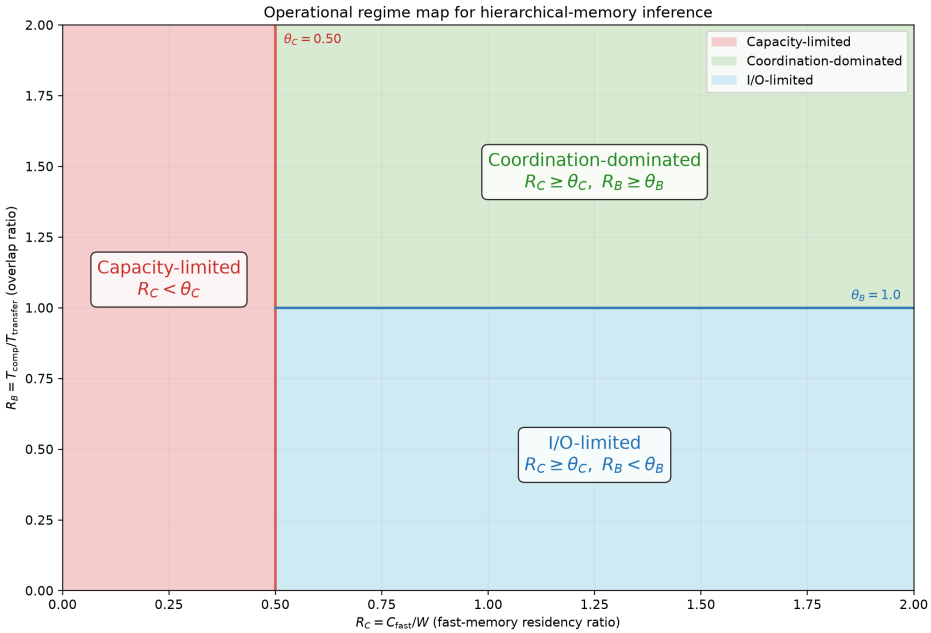

Fig. 1 Analytical regime map. The vertical line is the declared majority-residency convention $\theta_C = 0.50$; the horizontal segment is the derived overlap boundary $\theta_B = 1.0$. The diagram is generated from the same constants as the released classifier and contains no empirical data.

where C_{fast} is usable fast-memory capacity, W is the active working set, D is compulsory cross-tier traffic per step, B_{slow} is sustained bandwidth on the limiting link, and $T_{\text{transfer}} = D/B_{\text{slow}}$. Unlike a ratio that uses total step time, R_B is not circular: it compares independently timed computation and transfer intervals.

We use $\theta_C = 0.50$ as an explicit majority-residency convention and derive $\theta_B = 1$ from equality of computation and transfer time. These definitions produce three operational labels:

- **capacity-limited**: $R_C < \theta_C$;
- **I/O-limited**: $R_C \geq \theta_C$ and $R_B < \theta_B$; and
- **coordination-dominated**: $R_C \geq \theta_C$ and $R_B \geq \theta_B$.

The capacity boundary is a convention, not a thermodynamic critical point. The I/O boundary is an overlap boundary: for $R_B < 1$, at least $T_{\text{transfer}} - T_{\text{comp}}$ cannot be hidden by ideal overlap.

2.2 Irreducible costs and falsifiable predictions

For accounting purposes, write a step as

$$T_{\text{total}} = T_{\text{comp}} + T_{\text{mem}} + T_{\text{swap}} + T_{\text{sync}}, \tag{2}$$

Table 1 Selected measured points from the compiled CPU pointer-chain experiment. Values are mean ± one standard deviation over seven trials.

Working set (MiB)	R_C	Latency (ns/load)	Interpretation
0.5	96.000	6.8 ± 0.1	small working set
48	1.000	118.5 ± 6.6	reported LLC capacity
96	0.500	132.5 ± 14.3	residency convention
256	0.188	169.7 ± 10.4	largest working set

a conservative service-time lower bound is

$$T_{\text{total}} \geq \max\left\{T_{\text{comp}}, \rho W \max(0, 1 - R_C), \frac{D}{B_{\text{slow}}}\right\}, \tag{3}$$

where ρ is a lower bound on service time per non-resident byte for a specified access pattern. Taking the maximum permits ideal overlap and avoids double-counting bytes that contribute to both a cache miss and a cross-tier transfer. Synchronisation and contention can only increase observed time, so Equation (3) is not a performance predictor.

The formulation makes two falsifiable predictions. First, reducing residency while holding the access pattern fixed should expose a higher memory-access cost. Second, when $R_C \geq \theta_C$, configurations with $R_B < 1$ must expose transfer time even under ideal compute–transfer overlap. It does not, without additional measurements, predict the magnitude of a strategy's speed-up or claim that strategy rankings necessarily invert.

2.3 CPU proof-of-concept measurements

We evaluated working-set sensitivity on an Intel Xeon Platinum 8370C CPU exposed through a virtualised Linux environment. The experiment used a randomised pointer chain over working sets from 0.5 to 256 MiB and seven trials per point. A compiled C loop performed dependent loads and timed them with `CLOCK_MONOTONIC_RAW`; Python was used only to construct the chain and summarise trials. Linux sysfs reported a 48 MiB last-level cache, which we used as C_{fast}. Mean dependent-load time was 6.8 ± 0.1 ns at 0.5 MiB ($R_C = 96$), 118.5 ± 6.6 ns at 48 MiB ($R_C = 1$), 132.5 ± 14.3 ns at 96 MiB ($R_C = 0.5$), and 169.7 ± 10.4 ns at 256 MiB ($R_C = 0.188$), where uncertainty is one standard deviation across trials. The non-monotonic intermediate points show that cache level, translation, virtualisation, and system noise matter. In particular, these data do not show a discontinuity at the conventional $R_C = 0.5$ label.

As a separate harness check, we used the layered matrix program with 12 layers, $d = 512$, batch size 8, and five timing windows. It varied the fraction of layer weights retained in the fast tier. The mean total latency averaged over the three sampled points below $R_C = 0.5$ was 4.03 ms, compared with 1.87 ms for the fully resident point, a 2.15-fold descriptive ratio. Because computation and copying share the same CPU memory system, the accompanying R_B sweep was noisy and is not used as evidence for the I/O boundary.

These measurements are proof-of-concept results from one CPU, not a substitute for the previously planned A100, MI250, TPU v4, Inferentia2, and Optane campaign. Accordingly, we make no quantitative cross-accelerator, classifier- accuracy, energy, or strategy-inversion claim in this version.

2.4 Reproducibility boundary

The repository contains the raw JSONL records, machine-readable summaries, and scripts used for Table 1. It also contains a simulated backend for testing the analysis pipeline. Simulated output is labelled as such and is not used as empirical evidence. Accelerator support in the measurement harness is an executable protocol for future validation; it is not evidence that those accelerators were measured in this study.

3 Discussion

The two-ratio description separates two questions that are often conflated in hierarchical-memory optimisation: whether the active data can reside in the fast tier, and whether compulsory transfer can be hidden behind computation. The resulting labels are operational rather than universal phases. $\theta_B = 1$ follows exactly from the overlap definition, whereas $\theta_C = 0.5$ is a declared midpoint that makes "majority resident" precise. A different application may choose another residency threshold without changing Equation (3).

The CPU probes show that latency depends strongly on working-set residency, but they do not validate a sharp boundary at $R_C = 0.5$. They stress different mechanisms: dependent pointer loads probe cache and translation effects, while the layered matrix harness combines weight copying with computation. Neither test emulates accelerator DMA, asynchronous streams, HBM, or collective communication. The data therefore do not establish the I/O crossover on a GPU or the generality of the three labels across model families.

The framework is most useful as a measurement discipline. Reporting R_C and R_B alongside latency makes hidden assumptions about residency and overlap inspectable. It also prevents a common circularity: using total step duration in both the numerator of a bandwidth ratio and as the outcome being explained. For deployment, C_{fast}, W, D, sustained bandwidth, and isolated compute time must be measured for the actual workload and hardware rather than copied from nominal data-sheet values.

Several limitations are material. The present evidence comes from one CPU and a small synthetic probe; it does not include production models, concurrent requests, accelerator energy measurements, strategy comparisons, or unseen- hardware classifier tests. The standard deviations describe repeated trials on one machine and are not confidence intervals over a population of platforms. The pointer-chain endpoints also span several hierarchy transitions, so their ratio should not be interpreted as a universal effect size. Finally, Equation (2) is an accounting identity only when its terms are defined as non-overlapping critical-path contributions; the max-form lower bound avoids assuming that decomposition in measured systems.

A decisive follow-up should preregister operating-point grids around $R_C = 0.5$ and $R_B = 1$, collect raw event traces on at least two accelerator architectures, and compare fixed orchestration policies without changing other workload parameters. Such data could test boundary sharpness, policy ranking, and generalisation. Until those measurements are available, the contribution of this work is the corrected analytical formulation, open protocol, and CPU proof of concept rather than a claim of universal accelerator validation.

4 Methods

Definitions and classification

We calculate $R_C = C_{\text{fast}}/W$ from a declared usable fast-tier capacity and active working-set size. We calculate $R_B = T_{\text{comp}}/T_{\text{transfer}}$, with compute and transfer timed separately. Classification gives capacity precedence: $R_C < 0.5$ is capacity-limited; otherwise $R_B < 1$ is I/O-limited; all remaining points are coordination-dominated. This rule is implemented in the released Python code.

Pointer-chain capacity probe

The CPU probe allocates a NumPy array and constructs a seeded random permutation. A C11 kernel compiled with `-O3` traverses it as a dependent pointer chain; each loaded index determines the address of the next load, limiting vectorisation, instruction-level parallelism, and prefetching. We measured working sets of 0.5, 1, 2, 4, 8, 16, 24, 32, 48, 64, 96, 128, 192, and 256 MiB. Each point used seven timed trials after warm-up. The output stores working-set size, R_C, mean nanoseconds per dependent load, standard deviation, and trial count. The fast-tier capacity was read from Linux sysfs and was 48 MiB. The host exposed three logical CPUs of an Intel Xeon Platinum 8370C under KVM; the summary records the kernel, Python, and NumPy versions. Timing used `clock_gettime(CLOCK_MONOTONIC_RAW)` inside the compiled loop. No outlier was removed and CPU affinity or frequency was not controlled.

Layered matrix and copy probe

The second probe allocates 12 square, float32 layer matrices with $d = 512$ and batch size 8. A selected fraction remains resident; non-resident matrices are copied before multiplication. NumPy's wall-clock path was used on the CPU, with five windows per point. Matrix entries are deterministically seeded and scaled by $1/\sqrt{d}$ to prevent numerical overflow during repeated multiplication. The script also supports CUDA-event timing and XLA synchronisation, but no GPU or TPU result is reported here.

For the reported capacity ratio, we averaged total latency at the three sampled points below $R_C = 0.5$ and divided by latency at $R_C = 1$. This endpoint summary is descriptive; no hypothesis test or population-level confidence interval is claimed. The pointer-chain table reports mean and population standard deviation across the seven repeated trials exactly as stored in the JSON summary.

Software and provenance

Both probes were run with Python 3.11 and NumPy. Seeds, grids, timing code, raw records, summaries, and commands are version controlled. The analytical regime map reads threshold constants from the same module as the classifier. The simulated backend is provided solely for software testing and qualitative exploration; its outputs are excluded from the reported measurements.

Statistics and reporting

This proof-of-concept study did not use random assignment, blinding, or a sample-size calculation. Trial counts were fixed before summarisation. We report all measured grid points in the machine-readable data, selected points in Table 1, arithmetic means, and one standard deviation. No claim of statistical significance is made. Hardware-platform generalisation requires independent machines and is left for future work.

Data availability

All data supporting the reported CPU measurements are included in the public repository at https://github.com/leemgs/orion, under `code/results/cpu_probe/` and `code/results/colab_probe/`. These directories contain the raw JSONL records and machine-readable JSON summaries used in the text and table. No accelerator data are reported in this article.

Code availability

The analysis and measurement code is available at https://github.com/leemgs/orion. The `code/experiments/` directory contains the CPU probes, analytical-figure generator, and optional CUDA/XLA measurement paths. Exact reproduction commands are provided in the README and Supplementary Information. A separately labelled simulated backend is supplied for software testing; simulated output is not used as evidence in this article.

Author Contributions

G.L.: Conceptualization, Formal analysis, Investigation, Methodology, Software, Validation, Visualization, Writing – original draft, Writing – review & editing.

Competing Interests

The author declares no competing interests.

Acknowledgements

We thank Donghyeon Kang, Seungmin Oh and Yunsu Lee for valuable feedback. We are also grateful to Wooyeon Lee, Hyoyoung Kim, and Heekuk Lee, experts in systems and memory, whose insights and meaningful comments greatly improved this work.

References

[1] Xiaoyu Ma and David Patterson. Challenges and research directions for large language model inference hardware. *IEEE Computer*, 59, 2026. URL https://arxiv.org/abs/2601.05047. Accepted.

[2] Amir Gholami, Zhewei Yao, Sehoon Kim, Coleman Hooper, Michael W. Mahoney, and Kurt Keutzer. Ai and memory wall. *IEEE Micro*, 2024. URL https://arxiv.org/abs/2403.14123.

[3] Zixuan Zhou, Xuefei Ning, Ke Hong, Tianyu Fu, Jiaming Xu, Shiyao Li, Yuming Lou, Luning Wang, Zhihang Yuan, Xiuhong Li, Shengen Yan, Guohao Dai, Xiao-Ping Zhang, Yuhan Dong, and Yu Wang. A survey on efficient inference for large language models. *arXiv preprint arXiv:2404.14294*, 2024. URL https://arxiv.org/abs/2404.14294.

[4] Ying Sheng, Lianmin Zheng, Binhang Yuan, Zhuohan Li, Max Ryabinin, Daniel Y. Fu, Zhiqiang Xie, Beidi Chen, Clark Barrett, Joseph E. Gonzalez, Percy Liang, Christopher Ré, Ion Stoica, and Ce Zhang. Flexgen: High-throughput generative inference of large language models with a single gpu. In *Proceedings of the 40th International Conference on Machine Learning (ICML)*, 2023. URL https://arxiv.org/abs/2303.06865.

[5] Reza Yazdani Aminabadi, Samyam Rajbhandari, Ammar Ahmad Awan, Chenyang Li, Dong Li, Erich Zheng, Olatunji Ruwase, and Yuxiong He. Deepspeed inference: Enabling efficient inference of transformer models at unprecedented scale. In *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis (SC)*, 2022. URL https://arxiv.org/abs/2207.00032.

[6] Chien-Chin Huang, Gu Jin, and Jinyang Li. Swapadvisor: Pushing deep learning beyond the gpu memory limit via smart swapping. In *Proceedings of the 25th International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, 2020. URL https://doi.org/10.1145/3373376.3378530.

[7] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E. Gonzalez, Hao Zhang, and Ion Stoica. Efficient memory management for large language model serving with pagedattention. *arXiv preprint arXiv:2309.06180*, 2023. URL https://arxiv.org/abs/2309.06180.

[8] R. L. Mattson, J. Gecsei, D. R. Slutz, and I. L. Traiger. Evaluation techniques for storage hierarchies. *IBM Systems Journal*, 9(2):78–117, 1970. doi: 10.1147/sj.92.0078. URL https://doi.org/10.1147/sj.92.0078.

[9] Samuel Williams, Andrew Waterman, and David Patterson. Roofline: an insightful visual performance model for multicore architectures. *Communications of the ACM*, 52(4):65–76, 2009. doi: 10.1145/1498765.1498785. URL https://doi.org/10.1145/1498765.1498785.